SIMATS ENGINEERING

PROGRAMMING IN JAVA (CSA0923)

ASSIGNMENT 4

Name: M. Naga Shalini

Register Number: 192524183

TASK 1 – GUI DESIGN REVIEW

Task 1.1 – Diagnose the Design

• All button actions are mixed in one listener, making the code difficult to read, test and maintain.

• The action command is compared using ==. String contents should be compared using equals().

• The amount is parsed without checking empty or invalid input. A non-numeric value can cause NumberFormatException.

• Database access and Thread.sleep() on the event-dispatch thread can freeze the GUI.

• Printing “Paid” only to the console gives the user no on-screen confirmation.

• The Receipt button needs its own implementation.

• SQL concatenation can cause SQL injection; PreparedStatement should be used.

• Window closing and database connection cleanup must be handled. Checked exceptions must also be handled.

Task 1.2 – Programiz Single-File Demonstration

This console version demonstrates payment validation, successful payment, clearing fields and receipt generation. It uses an in-memory example instead of MySQL because Programiz is not connected to the college database.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
```

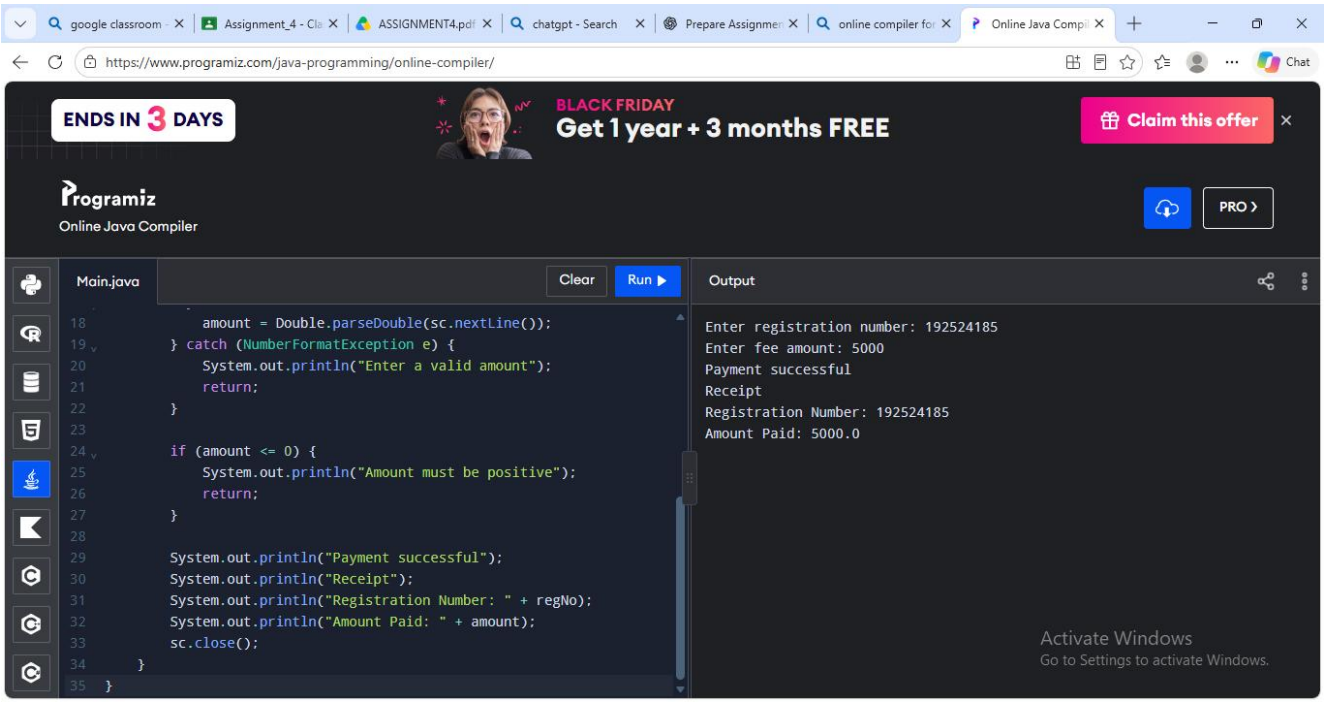
```
        Scanner sc = new Scanner(System.in);
        String regNo;
        double amount = 0;
        System.out.print("Enter registration number: ");
        regNo = sc.nextLine().trim();

        if (regNo.isEmpty()) {
            System.out.println("Enter registration number");
            return;
        }

        System.out.print("Enter fee amount: ");
        try {
            amount = Double.parseDouble(sc.nextLine());
        } catch (NumberFormatException e) {
            System.out.println("Enter a valid amount");
            return;
        }

        if (amount <= 0) {
            System.out.println("Amount must be positive");
            return;
        }

        System.out.println("Payment successful");
        System.out.println("Receipt");
        System.out.println("Registration Number: " + regNo);
        System.out.println("Amount Paid: " + amount);
        sc.close();
    }
}
```



Task 1.3 – Design Defence

The redesigned structure separates the interface from the service logic. The interface collects input and displays messages, while FeeService validates the registration number and amount and performs the payment. Separate responsibilities make the program easier to test and maintain. In a real application, database work should use PreparedStatement and run in SwingWorker so that the event-dispatch thread is not blocked.

Alternative Concept and Complexity Analysis

A single listener with an if–else chain may compare up to k button names, giving O(k) dispatch in the worst case. Separate handlers route an event directly to its action, approximately O(1) dispatch, but require O(k) listener storage. Separate handlers are easier to maintain when the number of controls grows.

TASK 2 – FROM REQUIREMENT TO EVENT-DRIVEN DESIGN

Task 2.1 – Model the Events

Book button – ActionEvent – ActionListener – validate details and book appointment.

Department combo box – ItemEvent – ItemListener – refresh the doctor list.

Doctor combo box – ItemEvent – ItemListener – update selected doctor.

Time-slot combo box – ItemEvent – ItemListener – update selected slot.

Window close button – WindowEvent – WindowListener/WindowAdapter – ask for confirmation if booking is in progress.

Patient ID field (Enter) – ActionEvent – ActionListener – start booking validation.

Task 2.2 – Component Hierarchy

• JFrame: Appointment Desk

• JPanel (BorderLayout)

• JPanel (GridLayout) – Form

• JLabel + JTextField – Patient ID

• JLabel + JComboBox – Department, Doctor and Time slot

• JButton – Book Appointment

• Status area – Confirmation / Error label

The JFrame is the main window. Panels group related controls. GridLayout aligns the form rows, while BorderLayout separates the form and status area.

Task 2.3 – Programiz Event-Logic Demonstration

This single-file program simulates department selection, doctor-list refresh and appointment booking using console input.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Map<String, List<String>> doctors = new HashMap<>();
        doctors.put("Cardiology", Arrays.asList("Dr Ravi", "Dr
Meena"));
        doctors.put("Dental", Arrays.asList("Dr Kumar", "Dr
Priya"));
        doctors.put("General", Arrays.asList("Dr Arun", "Dr
Sara"));

        System.out.print("Enter patient ID: ");
        String patient = sc.nextLine().trim();
```

```
        if (patient.isEmpty()) {
            System.out.println("Enter patient ID");
            return;
        }

        System.out.println("Departments: Cardiology, Dental,
General");
        System.out.print("Select department: ");
        String dept = sc.nextLine().trim();
        List<String> list = doctors.get(dept);

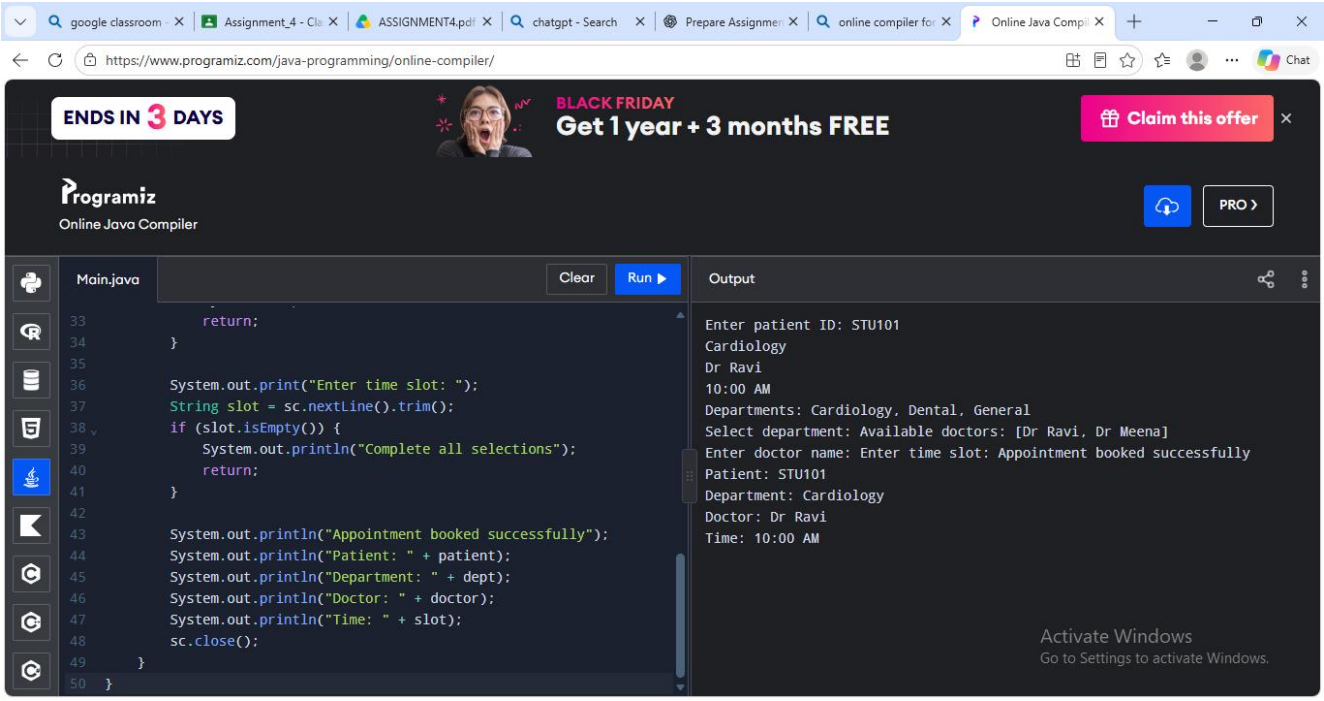
        if (list == null) {
            System.out.println("No doctors available");
            return;
        }

        System.out.println("Available doctors: " + list);
        System.out.print("Enter doctor name: ");
        String doctor = sc.nextLine().trim();
        if (!list.contains(doctor)) {
            System.out.println("Select a valid doctor");
            return;
        }

        System.out.print("Enter time slot: ");
        String slot = sc.nextLine().trim();
        if (slot.isEmpty()) {
            System.out.println("Complete all selections");
            return;
        }

        System.out.println("Appointment booked successfully");
        System.out.println("Patient: " + patient);
        System.out.println("Department: " + dept);
        System.out.println("Doctor: " + doctor);
        System.out.println("Time: " + slot);
```

```
            sc.close();
        }
}
```



Task 2.4 – Trace the Event Sequence

The main method creates the appointment window on the event-dispatch thread. The JFrame constructor creates components and listeners. setVisible(true) displays the window. Selecting a department generates an ItemEvent and refreshes the doctor list. Clicking Book generates an ActionEvent. The handler validates the input and saves the appointment. When the close button is pressed, windowClosing() runs and may show a confirmation dialog.

Applet comparison: an applet uses init(), start(), paint(), stop() and destroy(). A JFrame is created through its constructor and shown using setVisible(true); its window events handle closing. A desktop JFrame does not use applet life-cycle methods.

Alternative Concept and Complexity Analysis

If doctors are stored in Map<String, List<String>>, average department lookup is O(1), followed by O(r) work to display r doctors. Memory is O(d + m), where d is the number of departments and m is the number of doctor records. Querying a database on every department change adds database latency and more requests.

TASK 3 – FROM SKETCH TO LAYOUT

Task 3.1 – Choose the Layout Managers

Main JFrame: BorderLayout places the search row at NORTH, results at CENTER and bottom controls at SOUTH. Search panel: FlowLayout keeps the text field, category and Search button in one row. Results panel: JTable inside JScrollPane occupies CENTER and grows when resized. Bottom panel: FlowLayout arranges Prev, Next and Reserve buttons.

Task 3.2 – Nested Panel Structure

• JFrame – BorderLayout

• NORTH: JPanel – FlowLayout – search field, category and Search button

• CENTER: JScrollPane containing JTable

• SOUTH: JPanel – Prev, Next, Reserve and status label

Task 3.3 – Programiz Search Demonstration

The following single-file console version demonstrates searching a small sample library list. The actual JTable and resizing layout cannot be shown in Programiz.

```
import java.util.*;

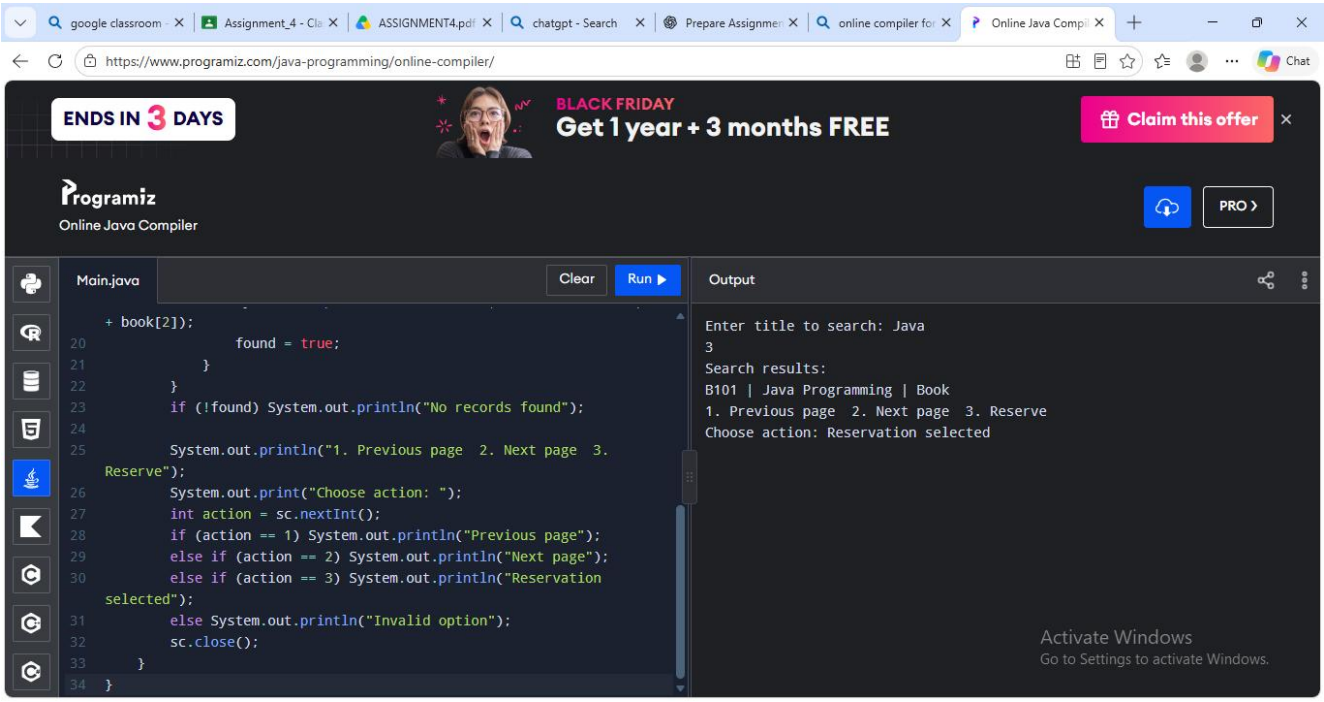
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[][] books = {
            {"B101", "Java Programming", "Book"},
            {"B102", "Database Systems", "Book"},
            {"J201", "Science Review", "Journal"}
        };

        System.out.print("Enter title to search: ");
        String key = sc.nextLine().toLowerCase();
        System.out.println("Search results:");

        boolean found = false;
        for (String[] book : books) {
            if (book[1].toLowerCase().contains(key)) {
```

```
                System.out.println(book[0] + " | " + book[1] +
" | " + book[2]);
                found = true;
            }
        }
        if (!found) System.out.println("No records found");

        System.out.println("1. Previous page  2. Next page  3.
Reserve");
        System.out.print("Choose action: ");
        int action = sc.nextInt();
        if (action == 1) System.out.println("Previous page");
        else if (action == 2) System.out.println("Next page");
        else if (action == 3) System.out.println("Reservation
selected");
        else System.out.println("Invalid option");
        sc.close();
    }
}
```



Task 3.4 – Resizing Behaviour

The original GUI must be tested at approximately 500 × 300, 650 × 400 and 900 × 600 pixels. BorderLayout gives extra space to CENTER, so the JTable scroll area grows.

Programiz cannot create or resize this desktop window; therefore, the three GUI screenshots cannot be captured from Programiz.

Alternative Concept and Complexity Analysis
GridBagLayout supports flexible forms but needs constraints for each component. Absolute positioning with setLayout(null) and setBounds() can cause overlap or clipping when the window is resized. Nested BorderLayout and FlowLayout are generally easier to maintain. A layout pass processes components in approximately O(n) time for n components.

TASK 4 – EVENT-HANDLING FORENSICS

Task 4.1 – Locate the Defect
The method name mouseclicked is incorrect. The correct MouseAdapter method is mouseClicked with a capital C. Java is case-sensitive, so the misspelled method does not override the listener method. The label must also be updated after each click.

Task 4.2 – Construct a Failing Interaction
Drawing directly with getGraphics() is temporary. When a window is minimised and restored, the operating system may repaint the window and the dots disappear. The points should be stored and redrawn in paint().

Task 4.3 – Programiz Stored-Point Demonstration
This console version demonstrates storing click coordinates and retaining them in a list. Programiz cannot capture mouse clicks on a drawing canvas or demonstrate minimising/restoring a GUI window.

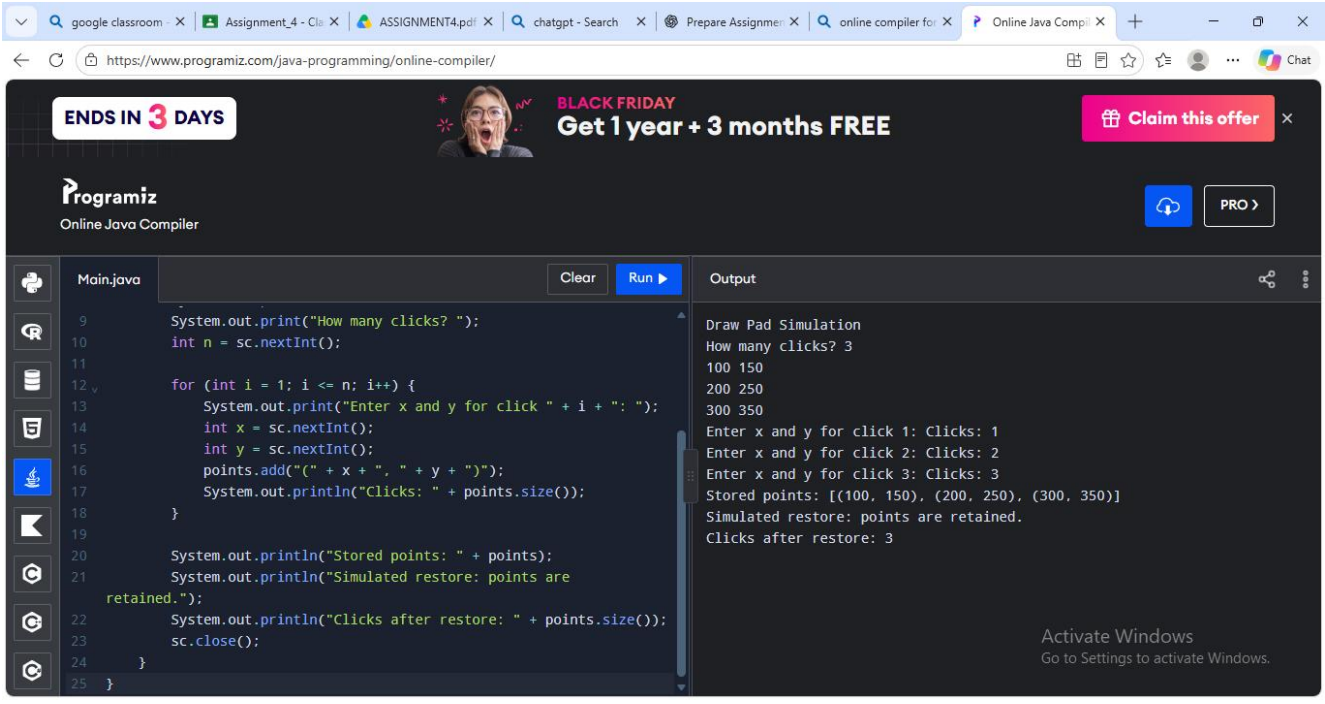
```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<String> points = new ArrayList<>();

        System.out.println("Draw Pad Simulation");
        System.out.print("How many clicks? ");
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
```

```
            System.out.print("Enter x and y for click " + i +
": ");
            int x = sc.nextInt();
            int y = sc.nextInt();
            points.add("(" + x + ", " + y + ")");
            System.out.println("Clicks: " + points.size());
        }

        System.out.println("Stored points: " + points);
        System.out.println("Simulated restore: points are
retained.");
        System.out.println("Clicks after restore: " +
points.size());
        sc.close();
    }
}
```



Task 4.4 – Prevent Recurrence

The annotation asks the compiler to verify that a method overrides a superclass or interface method. If mouseClicked is misspelled as mouseclicked while @Override remains, the compiler reports that the method does not override or implement a method from a supertype. Restore the correct spelling after the demonstration.

Correct handler:

```
@Override
public void mouseClicked(MouseEvent e) {
    points.add(e.getPoint());
    status.setText("Clicks: " + points.size());
    repaint();
}
```

Drawing stored points:

```
for (Point point : points) {
    g.fillOval(point.x - 3, point.y - 3, 6, 6);
}
```

Alternative Concept and Complexity Analysis

An alternative is to draw onto an off-screen BufferedImage and copy that image in paint(). With n stored points, redrawing the list takes O(n) time and O(n) storage. A BufferedImage uses O(width × height) pixel storage and can make repeated repainting faster.

TASK 5 – SAME SCREEN, DIFFERENT EVENT-HANDLING STYLES

Task 5.1 – Four Implementations

The same converter can use four event-handling styles: (a) frame implements ActionListener, (b) named inner class, (c) anonymous inner class and (d) lambda expression. The original assignment provides a complete lambda implementation. The console program below demonstrates its conversion and validation logic.

Task 5.2 – Change the Requirement

To convert when Enter is pressed in a Swing application, register the same ActionListener on the amount field using amount.addActionListener(handler). To show a result dialog, use JOptionPane.showMessageDialog(this, message). Reusing one handler avoids duplicated conversion logic.

Task 5.3 – Programiz Single-File Converter

The sample exchange rates below are the assignment's sample values, not live currency rates.

```
import java.util.Scanner;
```

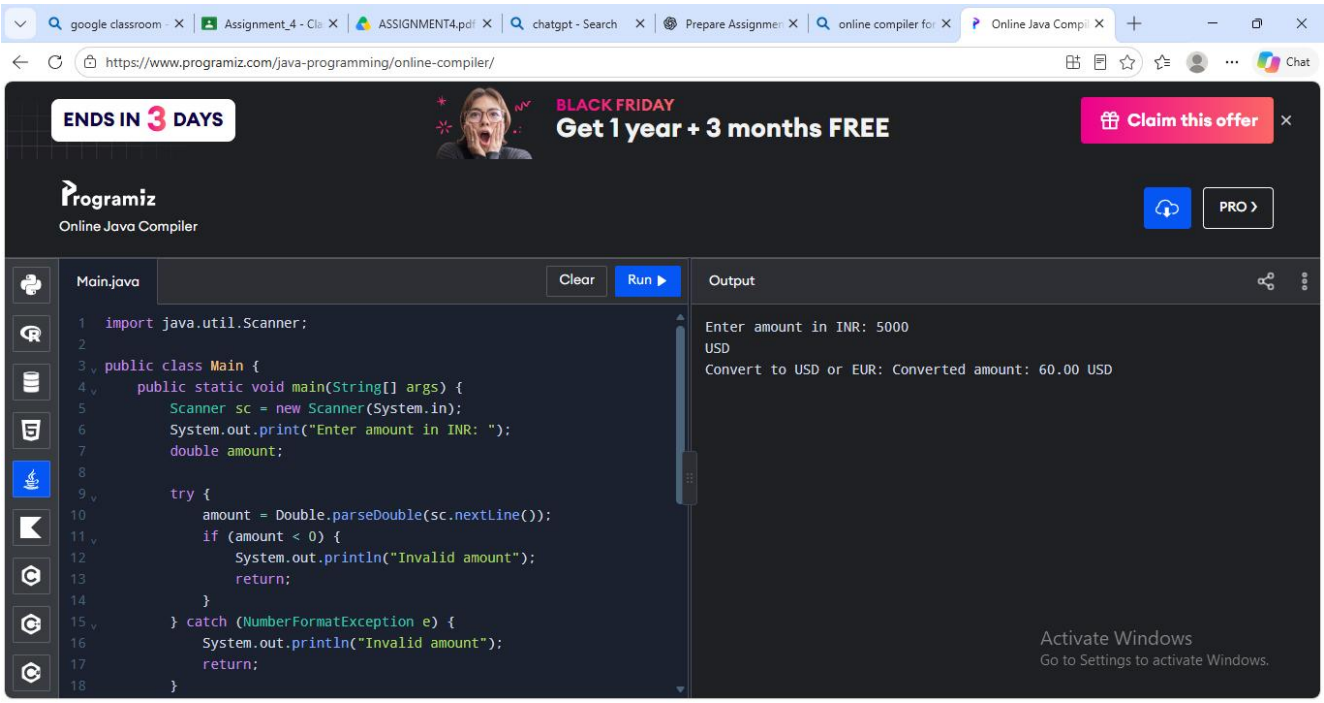
```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter amount in INR: ");
        double amount;

        try {
            amount = Double.parseDouble(sc.nextLine());
            if (amount < 0) {
                System.out.println("Invalid amount");
                return;
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid amount");
            return;
        }

        System.out.print("Convert to USD or EUR: ");
        String currency = sc.nextLine().trim().toUpperCase();
        double rate;

        if (currency.equals("USD")) rate = 0.012;
        else if (currency.equals("EUR")) rate = 0.011;
        else {
            System.out.println("Select USD or EUR");
            return;
        }

        double converted = amount * rate;
        System.out.printf("Converted amount: %.2f %s%n",
                          converted, currency);
        sc.close();
    }
}
```



Alternative Concept and Complexity Analysis

A method reference such as this::convert can refer to an existing method. One shared Action can be attached to a button, menu item and keyboard shortcut. Event dispatch is O(1) for each registered trigger. The main difference between the styles is readability, reuse and maintainability.

TASK 6 – DESIGN CHALLENGE: CAMPUS EVENT REGISTRATION

Stage 1 – Interpret the Problem

Students register for events. Committee members view, filter and export registrations. The application stores student ID, student name, email and event ID. Required functions include input validation, duplicate prevention, confirmation, viewing records, filtering and CSV export.

Stage 2 – Screen and Event Flow

- Open application
- Choose Register or View Registrations
- Enter details and select event
- Validate required fields and email
- Check duplicate registration

• Save registration and show confirmation/error

• View registrations and filter by event

• Export displayed records to CSV

Stage 3 – Critical Design Decisions
Input validation belongs in RegistrationService. Duplicate checking belongs in the service and database layer. A UNIQUE constraint on (student_id, event_id) prevents duplicate registrations. JDBC statements belong in RegistrationDAO. SwingWorker performs database work away from the event-dispatch thread. PreparedStatement protects user-supplied values.

Stage 4 – Classes and Handlers
RegistrationFrame handles the registration form, table model, event filter, register action, filter action and CSV export. RegistrationService validates data and registers students. RegistrationDAO inserts and retrieves database records. Registration stores registration ID, student ID, event ID and timestamp.

Stage 5 – Programiz Single-File Demonstration
Programiz cannot connect to the assignment's MySQL database or display the Swing JTable. This single-file version uses an in-memory list to demonstrate validation, duplicate checking, filtering and CSV-style output. Data is lost when the program ends.

```
import java.util.*;
public class Main {
    static class Registration {
        String id, name, email, event;
        Registration(String i, String n, String m, String e) {
            id = i; name = n; email = m; event = e;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<Registration> rows = new ArrayList<>();

        System.out.print("Student ID: ");
        String id = sc.nextLine().trim();
```

```
        System.out.print("Student name: ");
        String name = sc.nextLine().trim();
        System.out.print("Email: ");
        String email = sc.nextLine().trim();
        System.out.print("Event (TECH/CULTURE/SPORTS): ");
        String event = sc.nextLine().trim().toUpperCase();

        if (id.isEmpty() || name.isEmpty()) {
            System.out.println("Enter student ID and name");
            return;
        }
        if (!email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-
]+$")) {
            System.out.println("Enter a valid email");
            return;
        }
        if (!event.equals("TECH") && !event.equals("CULTURE")
                && !event.equals("SPORTS")) {
            System.out.println("Select a valid event");
            return;
        }

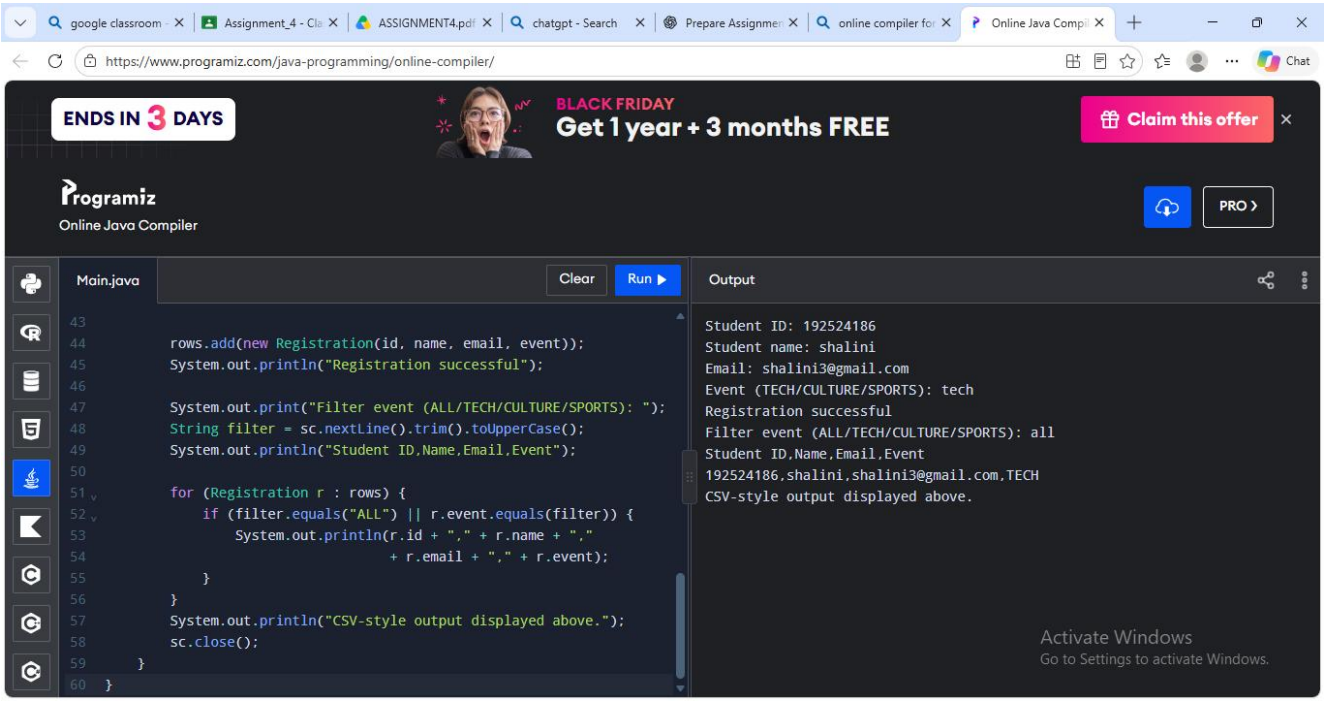
        for (Registration r : rows) {
            if (r.id.equals(id) && r.event.equals(event)) {
                System.out.println("Already registered for
this event");
                return;
            }
        }

        rows.add(new Registration(id, name, email, event));
        System.out.println("Registration successful");

        System.out.print("Filter event
(ALL/TECH/CULTURE/SPORTS): ");
        String filter = sc.nextLine().trim().toUpperCase();
```

```
        System.out.println("Student ID,Name,Email,Event");

        for (Registration r : rows) {
            if (filter.equals("ALL") || r.event.equals(filter))
{
                System.out.println(r.id + "," + r.name + ","
                                   + r.email + "," + r.event);
            }
        }
        System.out.println("CSV-style output displayed
above.");
        sc.close();
    }
}
```



MySQL Table – Original GUI Project Reference

The original desktop version uses MySQL. The following SQL creates the required table when running the full JDBC application outside Programiz:

```
CREATE DATABASE campus_events;
USE campus_events;

CREATE TABLE registrations (
```

```
    registration_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id VARCHAR(30) NOT NULL,
    student_name VARCHAR(100) NOT NULL,
    email VARCHAR(120) NOT NULL,
    event_id VARCHAR(30) NOT NULL,
    registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_student_event UNIQUE (student_id, event_id)
);
```

Alternative Concept and Complexity Analysis

Loading all N registrations requires O(N) loading work and O(N) row memory. Paging with LIMIT and OFFSET loads P rows per page, so the client stores O(P) rows. Database query time depends on indexes and offset size. SwingWorker keeps database work off the event-dispatch thread.

Execution evidence: Programiz can demonstrate registration validation, duplicate checking within the current run, filtering and CSV-style text. It cannot demonstrate persistent MySQL storage, actual duplicate-key handling, JTable filtering or saving a CSV file through a GUI file chooser.